

## **Assignment – 4**

**Name:** Vaddi Govardhan

**Reg No:** 142502032

**Github Link:**

[mlops2025w\\_142502032/Assignment/Assignment\\_4\\_at\\_main\\_vaddi-govardhan/mlops2025w\\_142502032](https://github.com/mlops2025w_142502032/Assignment/Assignment_4_at_main_vaddi-govardhan/mlops2025w_142502032)

### **Question-3:**

**CRUD operations on both approaches and write the differences you have observed regarding the performance and provide proof for your claims. (5 Marks)**

The evaluation was conducted on a uniform subset of the UCI Online Retail dataset containing over one thousand invoice-line items.

Two alternative persistence layers were implemented:

1. **Relational 2-NF design** (SQLite)
  - Five normalized tables: Country, Customer, Product, Invoice, InvoiceLine.
  - Foreign-key constraints and indexes on join keys.
2. **Document-oriented design** (MongoDB, transaction-centric)
  - One document per invoice, embedding all line-items.
  - Indexes on customerId and invoiceDate.

Both environments were exercised with comparable CRUD workloads:

- **Create:** insert 10 synthetic invoice-lines / embedded lines.
- **Read:** top-10 customers by revenue (SQL: GROUP BY join; Mongo: \$unwind+\$group).
- **Update:** apply 1 % discount to a single invoice.
- **Delete:** remove the synthetic test rows / document.

Timings were captured programmatically (Python time.perf\_counter) on the same host.

### **Observed Results:**

Measured wall-clock times (seconds):

| <b>Operation</b>               | <b>SQL (2NF SQLite)</b> | <b>MongoDB (document)</b> |
|--------------------------------|-------------------------|---------------------------|
| <b>Create</b>                  | 0.0077 s                | 0.0084 s                  |
| <b>Read (top-10 customers)</b> | 0.0021 s                | 0.0018 s                  |
| <b>Update (1 % discount)</b>   | 0.0057 s                | 0.0010 s                  |
| <b>Delete (test rows/doc)</b>  | 0.0056 s                | 0.0007 s                  |

#### **1. Insertion (C-operation)**

MongoDB's bulk insertMany is in principle optimized for high-throughput writes; however, in this limited micro-benchmark SQLite performed comparably, benefiting from an already open transaction and in-memory journaling.

#### **2. Read + Aggregation (R/A-operations)**

- The SQL engine exploited B-tree indexes and pre-computed join plans to answer the GROUP BY query efficiently ( $\approx 2$  ms).
- MongoDB's aggregation pipeline using \$unwind and \$group achieved similar latency ( $\approx 1.8$  ms), reflecting the advantage of pre-denormalised embedded lines that avoid join cost.

#### **3. Update / Delete (U / D-operations)**

- In the relational model, a discount update affects every qualifying row in InvoiceLine, incurring an index lookup and row-level update;  $\approx 5$ – $6$  ms on this dataset.
- In the document model, the same logical change modifies a single document (with all embedded lines), producing  $\approx 1$  ms latency — confirming that document locality favours entity-scoped modifications.

#### **4. Aggregation over temporal dimension**

SQL's relational grouping on a date expression was very fast ( $\approx 1.9$  ms) because the join depth was modest and indexes existed on Invoice.invoice\_date.

Equivalent aggregation in MongoDB was not recorded in this run, but prior literature suggests comparable performance provided an index on the time-key exists.

#### **5. Data-layout vs workload fit**

The transaction-centric document model minimises join overhead for invoice-level operations and bulk updates/deletes;

The 2-NF relational model is advantageous when queries span many invoices and customers, or when product descriptions are large and benefit from deduplication.

For production-scale deployments:

- MongoDB is often preferable for write-heavy, invoice-centric OLTP workloads requiring atomic invoice-level updates.
- Relational SQL remains advantageous for reporting, multi-dimensional analytics, and space-efficient storage.

## Proofs:

